

UNIVERSIDAD DE GRANADA

PERIFÉRICOS Y DISPOSITIVOS DE INTERFAZ HUMANA

CURSO 2025/2026

Práctica 6. Reconocimiento de imágenes

Autora:
Inés Ruiz Sánchez

5 de junio de 2026



**UNIVERSIDAD
DE GRANADA**

Índice

1. Introducción y objetivos	3
2. Cuestiones a resolver	3
3. Configuración del entorno de trabajo	4
3.1. Ejemplo de caras	5
4. Ejercicios propuestos	7
4.1. Ejercicio 1	7
4.1.1. Prueba con <i>Imagen 1</i>	8
4.1.2. Prueba con <i>Imagen 2</i>	10
4.2. Ejercicio 2	12
4.2.1. Prueba con <i>Video 2</i>	13
4.2.2. Prueba con <i>Video 3</i>	14
4.3. Ejercicio 3	15
4.3.1. Ejemplo de gatos	15
4.3.2. Resolución de ejercicio 3	16
4.3.3. Prueba con Video 1	16
4.3.4. Prueba con Video 2	17
4.4. Ejercicio 4	18
4.4.1. Detección en imagen	18
4.4.2. Prueba con imagen	19
4.4.3. Detección en video	21
4.4.4. Prueba con Video 4	23
4.4.5. Prueba con Video 5	23
5. Anexo	24
5.1. Fichero <i>ejemplo-fotos-1.py</i>	24
5.2. Fichero <i>ejercicio1.py</i>	24
5.3. Fichero <i>ejemplo2.py</i>	25
5.4. Fichero <i>ejemplo3.py</i>	26
5.5. Fichero <i>ejercicio2.py</i>	26
5.6. Fichero <i>ejemplo-gatos.py</i>	28
5.7. Fichero <i>ejercicio3.py</i>	28
5.8. Fichero <i>ejercicio4_img.py</i>	30
5.9. Fichero <i>ejercicio4_vid.py</i>	31

Índice de figuras

1. Proceso de instalación de Python en Windows.	4
2. Instalación de OpenCV.	4
3. Repositorio OpenCV	5
4. Imagen inicial	5
5. Ejecución de ejemplo	6
6. Ejecución de ejemplo con <code>scaleFactor = 1.05</code>	6
7. Imagen 1	7
8. Imagen 2	7
9. Resultado de <i>imagen1</i> con <code>scaleFactor=1.35</code>	9
10. Error de <i>imagen1</i> con <code>scaleFactor=1.0</code>	9
11. Falsos positivos en <i>imagen1</i> con <code>scaleFactor=1.01</code>	10
12. Resultado de <i>imagen2</i> con <code>scaleFactor=1.5</code>	10
13. Resultado de <i>imagen2</i> con <code>scaleFactor=1.35</code>	11
14. Captura del video 2	14
15. Captura del video 3	15
16. Imagen inicial de gatos	15
17. Imagen con gatos reconocidos	16

18.	Captura del vídeo 1	17
19.	Captura del vídeo 2	18
20.	Imagen de prueba ejercicio4	19
21.	Resultado ejercicio 4 con imagen	20
22.	Resultado ejercicio 4 con imagen (prueba 2)	21
23.	Resultado ejercicio 4 con imagen (prueba 3)	22

Listings

1.	Instalación de OpenCV.	4
2.	Librerías para el Ejercicio 1	8
3.	Comprobación de parametro	8
4.	Guardar imagen	8
5.	Orden de ejecución - Imagen 1	8
6.	Prueba 1 con Imagen 1	8
7.	Prueba 2 con Imagen 1	9
8.	Prueba 3 con Imagen 1	9
9.	Orden de ejecución - Imagen 2	10
10.	Prueba 1 con Imagen 2	10
11.	Prueba 2 con Imagen 2	11
12.	Extracción de propiedades del video	12
13.	Ruta de guardado	12
14.	Códec y extensión	12
15.	Frame a frame en ejercicio2	13
16.	Cerrar ventana	13
17.	Limpieza del búfer	13
18.	Orden de ejecución - Video 2	14
19.	Prueba 1 con Video 2	14
20.	Orden de ejecución - Video 3	14
21.	Prueba 1 con Video 3	14
22.	Ejemplo de gatos	15
23.	Orden de ejecución - Video 1	16
24.	Prueba 1 con Video 1	17
25.	Orden de ejecución - Video 2	17
26.	Prueba 2 con Video 2	18
27.	Búsqueda de caras y cuerpos	18
28.	Orden de ejecución para detectar cara y cuerpo en imagen	19
29.	Prueba 1 con imagen para ejercicio 4	20
30.	Prueba 2 con imagen para ejercicio 4	20
31.	Prueba 2 con imagen para ejercicio 4	20
32.	Procesamiento frame a frame de video para ejercicio 4	22
33.	Prueba 1 con video4 para ejercicio 4	23
34.	Prueba 1 con video4 para ejercicio 4	23
35.	Código completo del ejemplo1	24
36.	Código completo del ejercicio1	24
37.	Código completo del ejemplo2	25
38.	Código completo del ejemplo3	26
39.	Código completo del ejercicio2	26
40.	Código completo del ejemplo-gatos	28
41.	Código completo del ejercicio3	28
42.	Código completo del ejercicio4 con imágenes	30
43.	Código completo del ejercicio4 con video	31

1. Introducción y objetivos

Los **objetivos** concretos de esta práctica son:

- Conocer cómo funcionan las técnicas de reconocimiento de imágenes.
- Hacer un programa sencillo para realizar detección facial.

Repositorio en GitHub: <https://github.com/ruiz314/PDIH/tree/main/P6>.

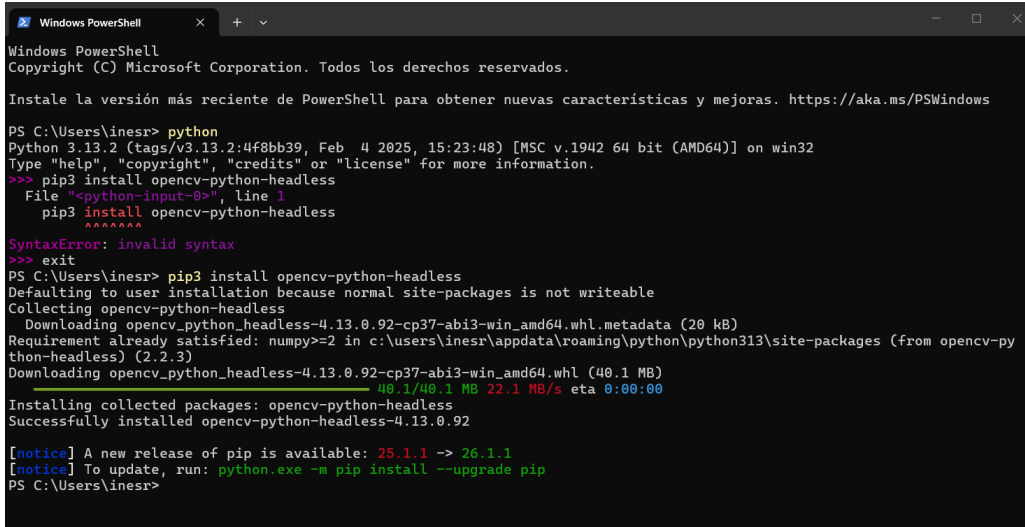
2. Cuestiones a resolver

Para superar esta práctica se proponen los siguientes ejercicios:

1. Hacer reconocimiento de caras de personas en imágenes (ejemplo básico).
2. Hacer reconocimiento de caras de personas en vídeos.
3. Hacer reconocimiento de caras de gatos en vídeos.
4. Hacer simultáneamente el reconocimiento de cuerpos y caras de personas en vídeos. Es requisito que ambas cosas se detenten al mismo tiempo. Como sugerencia, se puede hacer una primera aproximación para hacer la detección en imágenes y posteriormente en vídeos. Para este ejercicio se facilita un vídeo de ejemplo, aunque se pueden usar otros diferentes.

3. Configuración del entorno de trabajo

El sistema operativo usado es Microsoft Windows. A continuación, se detallan los pasos seguidos para la instalación del software, la gestión de entornos virtuales y la adquisición de los modelos de detección necesarios. Tenía instalada la **versión 3.13.2 de Python**, como se puede ver en la Figura 1.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\inesr> python
Python 3.13.2 (tags/v3.13.2:4f8bb39, Feb  4 2025, 15:23:48) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> pip3 install opencv-python-headless
File "<python-input-0>", line 1
    pip3 install opencv-python-headless
    ^^^^^^^
SyntaxError: invalid syntax
>>> exit
PS C:\Users\inesr> pip3 install opencv-python-headless
Defaulting to user installation because normal site-packages is not writeable
Collecting opencv-python-headless
  Downloading opencv_python_headless-4.13.0.92-cp37-abi3-win_amd64.whl.metadata (20 kB)
Requirement already satisfied: numpy>=2 in c:\users\inesr\appdata\roaming\python\python313\site-packages (from opencv-python-headless) (2.2.3)
Downloading opencv_python_headless-4.13.0.92-cp37-abi3-win_amd64.whl (40.1 MB)
  40.1/40.1 MB 22.1 MB/s eta 0:00:00
Installing collected packages: opencv-python-headless
Successfully installed opencv-python-headless-4.13.0.92

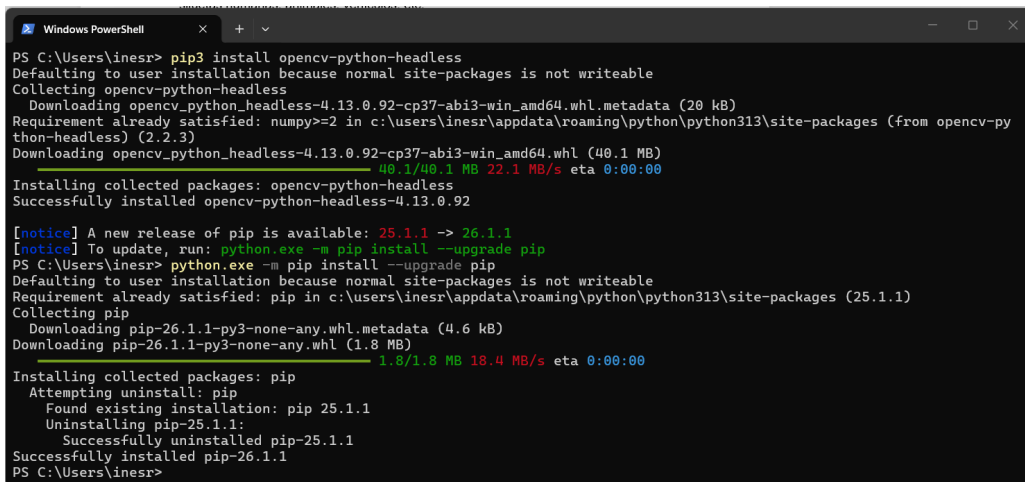
[notice] A new release of pip is available: 25.1.1 -> 26.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\inesr>
```

Figura 1: Proceso de instalación de Python en Windows.

Para la instalación de la Librería **OpenCV-Python** uso la siguiente orden:

```
pip3 install opencv-python-headless
```

Listing 1: Instalación de OpenCV.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\inesr> pip3 install opencv-python-headless
Defaulting to user installation because normal site-packages is not writeable
Collecting opencv-python-headless
  Downloading opencv_python_headless-4.13.0.92-cp37-abi3-win_amd64.whl.metadata (20 kB)
Requirement already satisfied: numpy>=2 in c:\users\inesr\appdata\roaming\python\python313\site-packages (from opencv-python-headless) (2.2.3)
Downloading opencv_python_headless-4.13.0.92-cp37-abi3-win_amd64.whl (40.1 MB)
  40.1/40.1 MB 22.1 MB/s eta 0:00:00
Installing collected packages: opencv-python-headless
Successfully installed opencv-python-headless-4.13.0.92

[notice] A new release of pip is available: 25.1.1 -> 26.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\inesr> python.exe -m pip install --upgrade pip
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: pip in c:\users\inesr\appdata\roaming\python\python313\site-packages (25.1.1)
Collecting pip
  Downloading pip-26.1.1-py3-none-any.whl.metadata (4.6 kB)
Downloading pip-26.1.1-py3-none-any.whl (1.8 MB)
  1.8/1.8 MB 18.4 MB/s eta 0:00:00
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 25.1.1
    Uninstalling pip-25.1.1:
      Successfully uninstalled pip-25.1.1
  Successfully installed pip-26.1.1
PS C:\Users\inesr>
```

Figura 2: Instalación de OpenCV.

Al estar trabajando con Windows, y ya tenía instalada la librería OpenCV de asignaturas previas, tenía algunos errores por lo que volví a la configuración en la que no se usa “headless”.

He accedido al repositorio oficial de código de OpenCV en GitHub (*opencv/data/haarcascades*) para descargar de forma local los clasificadores necesarios. En la Figura 3 se puede observar el contenido del repositorio. Para el ejercicio con reconocimiento de caras basta con descargar el fichero subrayado en naranja sobre rostros en posición frontal: *haarcascade_frontalface_alt.xml*. Para reconocer caras en videos hacen falta dos: *haarcascade_eye_tree_eyeglasses.xml* y *haarcascade_frontalface_default.xml*.

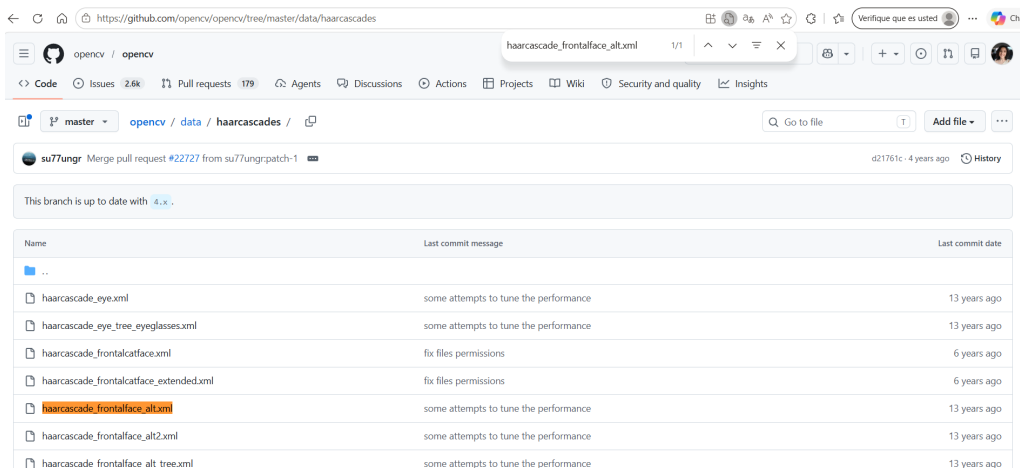


Figura 3: Repositorio OpenCV

3.1. Ejemplo de caras

El código fuente proporcionado por el profesor se encuentra en el Anexo 5.1. El material necesario (imagen y fichero *.xml*) están en la siguiente dirección del repositorio: <https://github.com/ruiz314/PDIH/blob/main/P6/ejemplos/caras/ejemplo-fotos-1.py>.

La imagen inicial es la que se muestra en la Figura 4. Muestra dos rostros de dos hombres en un primer plano. Al fondo de la imagen hay un cuadro con un rostro de otro hombre. La imagen está en blanco y negro.

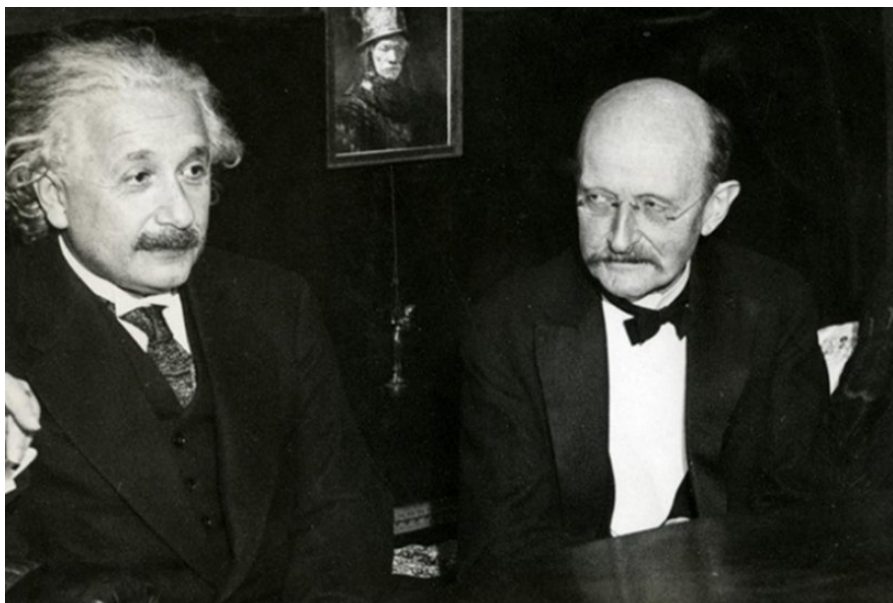


Figura 4: Imagen inicial

Para ejecutar este ejemplo en el que detecta las caras de una imagen basta con escribir *python* y el nombre del fichero:

```
python ejemplo-fotos-1.py
```

En la Figura 5 se puede observar el resultado de haber ejecutado el ejemplo proporcionado por el profesor. Solo se detecta una cara, la de la derecha (Max Planck). En un recuadro amarillo que he dibujado en mi terminal se puede ver el contenido de la carpeta, y subrayado por una línea amarilla viene el comando para ejecutar dicho programa.

En la Figura 6 se pueden ver dos recuadros rojos, es decir, el programa ha detectado dos caras en la imagen. La cara del cuadro no ha sido reconocida por el programa. Si reducimos el parámetro **scaleFactor** y le damos valor más cercano a 1,0 (p.ej. 1,05 en mi caso) veremos que sí identifica esa otra cara del cuadro. Subrayado en amarillo se encuentra el parámetro **scaleFactor**, para ver su valor en esa ejecución.

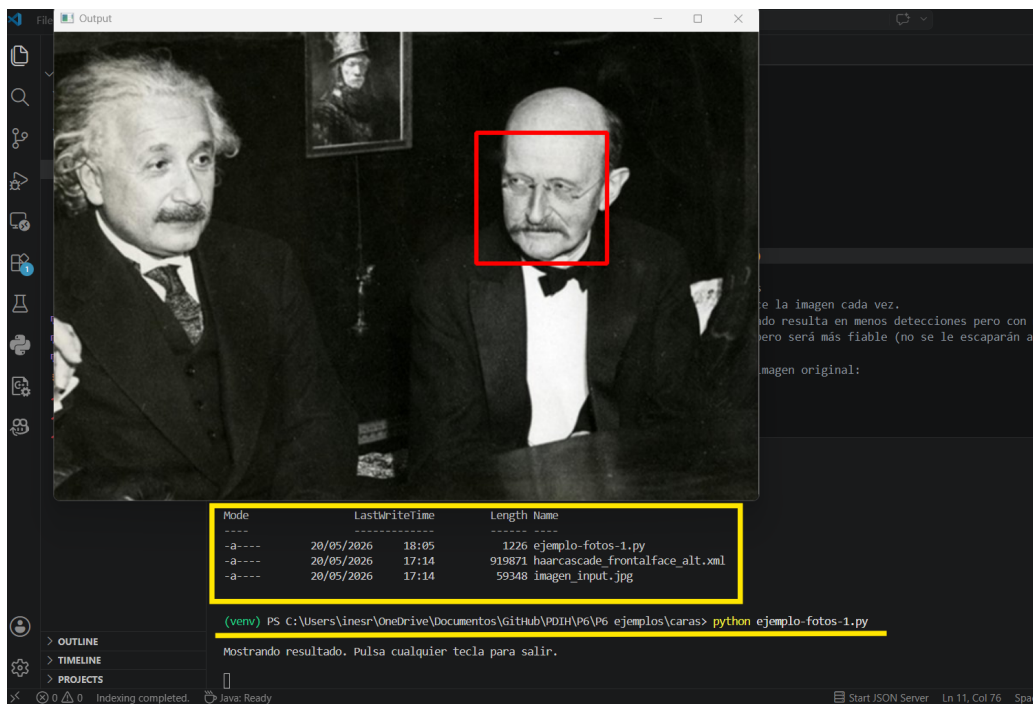


Figura 5: Ejecución de ejemplo

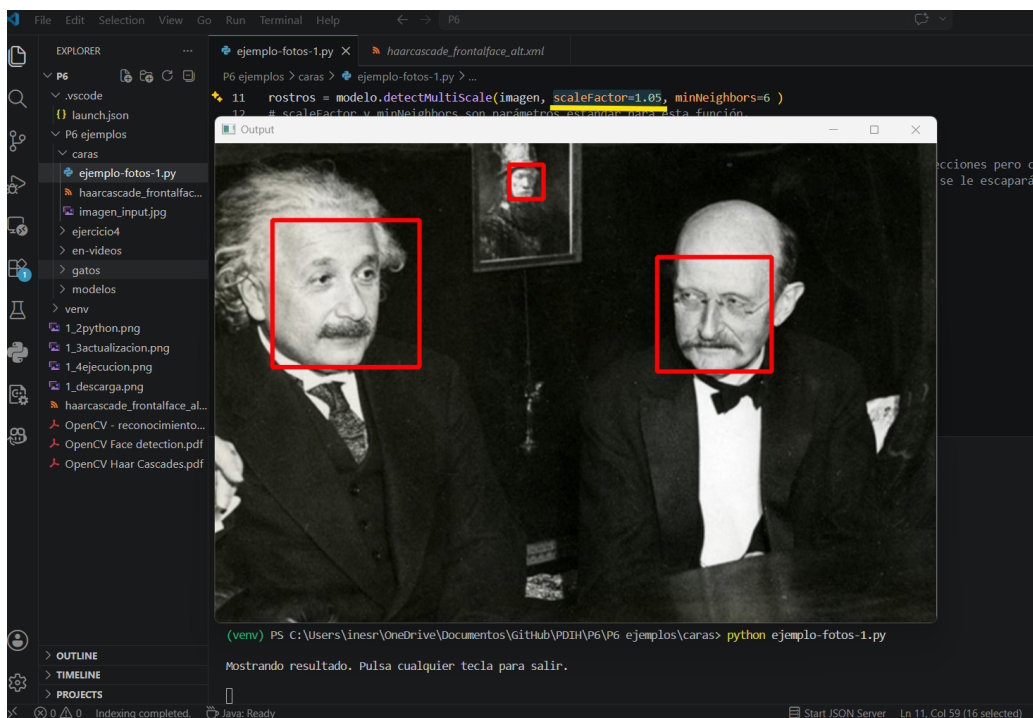


Figura 6: Ejecución de ejemplo con `scaleFactor = 1.05`

4. Ejercicios propuestos

4.1. Ejercicio 1

Para tener más imágenes con las que trabajar y probar el programa he buscado imágenes libres en la página *Pixabay*: <https://pixabay.com/es/photos/search/fotos%20de%20personas/>. A continuación presento las imágenes con las que voy a trabajar:

- Figura 7 - Dos personas en un parque. La persona de la izquierda lleva una gorra, tiene barba y está mirando de frente. La persona de la derecha tiene el pelo corto y casi está mirando de frente a la cámara.
- Figura 8 - Aparecen ocho personas, todas mirando a cámara y sonriendo.



Figura 7: Imagen 1



Figura 8: Imagen 2

Para esta tarea se ha empleado el clasificador frontal avanzado `haarcascade_frontalface_alt.xml`. También importo la librería `sys` para poder incluir la imagen con la que se trabaja como parámetro en la orden de

ejecución. Esto facilita probar con varias imágenes sin tener que tocar el fichero de código fuente. Además, uso la librería `os` para guardar las imágenes resultantes (con caras recuadradas) en la carpeta de trabajo.

```
1 import cv2
2 import sys
3 import os
```

Listing 2: Librerías para el Ejercicio 1

Como la imagen con la que se va a trabajar se pasa por parámetro hay que comprobar que se haya pasado, que la ruta exista y que no esté corrupta:

```
1 # Validación del número de argumentos en la línea de comandos
2 if len(sys.argv) < 2:
3     print("[ERROR] Uso correcto: python ejercicio1.py <nombre_imagen.jpg>")
4     sys.exit(1)
5
6 # Captura del parámetro del archivo de imagen
7 nombre_imagen = sys.argv[1]
8
9 #Cargamos nuestro modelo classificador de Haar:
10 modelo = cv2.CascadeClassifier('haarcascade_frontalface_alt.xml')
11
12 #Cargamos la imagen
13 img = cv2.imread(nombre_imagen)
14
15 # Control de excepciones ante rutas inexistentes o corruptas
16 if img is None:
17     print(f"[ERROR] No se pudo cargar la imagen '{nombre_imagen}'")
18     sys.exit(1)
```

Listing 3: Comprobación de parametro

A continuación, el programa sigue como el del ejemplo, haciendo el procesamiento de escala de grises, buscando rostros y dibujando los rectángulos sobre la imagen original. Una vez hecho eso, se guarda la imagen en disco separando el nombre de la imagen de la extensión:

```
1 # Separamos el nombre del archivo de su extensión (ej: 'foto' y '.jpg')
2 nombre_base, extension = os.path.splitext(nombre_imagen)
3
4 # Creamos el nuevo nombre para el archivo de salida
5 nombre_salida = f"resultado_{nombre_base}{extension}"
6
7 # Guardamos la imagen modificada en la carpeta de trabajo
8 cv2.imwrite(nombre_salida, img)
9 print(f"[INFO] Imagen resultante guardada con éxito como: {nombre_salida}")
```

Listing 4: Guardar imagen

4.1.1. Prueba con *Imagen 1*

Orden necesaria para la terminal:

```
(venv) PS C:\Users\inesr\PDIIH\P6> python .\ejercicio1.py .\imagen1.jpg
```

Listing 5: Orden de ejecución - Imagen 1

Prueba 1.1.

```
1 ...
2 rostros = modelo.detectMultiScale(imagen, scaleFactor=1.35, minNeighbors=6 )
3 ...
```

Listing 6: Prueba 1 con Imagen 1

El resultado usando `scaleFactor=1.35`, `minNeighbors=6` en la Figura 7 es la Figura 9. El motivo de usar `scaleFactor=1.35` es porque con el valor 1,5 no se reconoce ningún rostro. El rostro del hombre no lo llega a reconocer aunque se disminuya el valor de `scaleFactor`. Se puede observar que no reconoce falsos positivos.

Prueba 1.2.



Figura 9: Resultado de *imagen1* con `scaleFactor=1.35`

```
1 ...
2 rostros = modelo.detectMultiScale(imagen, scaleFactor=1.01, minNeighbors=6 )
3 ...
```

Listing 7: Prueba 2 con Imagen 1

Cuando disminuimos el `scaleFactor` a 1,0 sale un error porque el factor no puede tomar valores ≤ 1 . Ver Figura 10.

```
ejercicio1.py
P6 ejemplos > caras > ejercicio1.py
17 img = cv2.imread(nombre_imagen)
18
19 # Control de excepciones ante rutas inexistentes o corruptas
20 if img is None:
21     print(f"[ERROR] No se pudo cargar la imagen '{nombre_imagen}'")
22     sys.exit(1)
23
24 # Procesamiento en escala de grises
25 imagen = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
26
27 #Buscamos los rostros:
28 rostros = modelo.detectMultiScale(imagen, scaleFactor=1.0, minNeighbors=6 )
29
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(venv) PS C:\Users\inesr\OneDrive\Documents\Github\PDIH\P6\P6 ejemplos\caras> python ejercicio1.py imagen1.jpg
Traceback (most recent call last):
  File "C:\Users\inesr\OneDrive\Documents\Github\PDIH\P6\P6 ejemplos\caras\ejercicio1.py", line 28, in <module>
    rostros = modelo.detectMultiScale(imagen, scaleFactor=1.0, minNeighbors=6 )
cv2.error: OpenCV(4.13.0) D:\a\opencv-python\opencv-python\opencv\modules\objdetect\src\cascadefdetect.cpp:1389: error: (-215:Assertion failed) scaleFactor > 1 && _image.depth() == CV_8U in function 'cv::CascadeClassifierImpl::detectMultiScale'
(venv) PS C:\Users\inesr\OneDrive\Documents\Github\PDIH\P6\P6 ejemplos\caras>
```

Figura 10: Error de *imagen1* con `scaleFactor=1.0`

Prueba 1.3.

```
1 ...
2 rostros = modelo.detectMultiScale(imagen, scaleFactor=1.01, minNeighbors=6 )
3 ...
```

Listing 8: Prueba 3 con Imagen 1

Si tomamos `scaleFactor=1.01` para la Figura 7 no solo no se detecta la cara del señor de la izquierda, sino que hay falsos positivos, en los árboles y en el cuerpo de las dos personas de la imagen. Ver Figura 11.

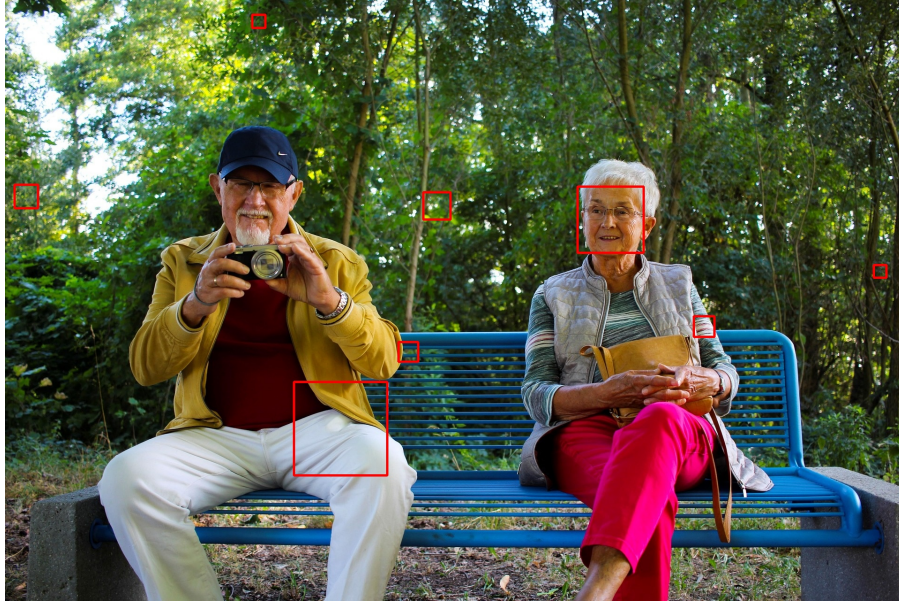


Figura 11: Falsos positivos en *imagen1* con `scaleFactor=1.01`

4.1.2. Prueba con *Imagen 2*

Orden necesaria para la terminal:

```
(venv) PS C:\Users\inesr\PDIH\P6> python .\ejercicio1.py .\imagen2.jpg
```

Listing 9: Orden de ejecución - Imagen 2

Prueba 2.1.

```
1 ...
2 rostros = modelo.detectMultiScale(imagen, scaleFactor=1.5, minNeighbors=6 )
3 ...
```

Listing 10: Prueba 1 con Imagen 2

El resultado usando `scaleFactor=1.5`, `minNeighbors=6` en la Figura 8 es la Figura 12. Se reconocen 5 rostros, de 8 que hay en la imagen. No hay falsos positivos.



Figura 12: Resultado de *imagen2* con `scaleFactor=1.5`

Prueba 2.2.

```
1 ...  
2 rostros = modelo.detectMultiScale(imagen, scaleFactor=1.35, minNeighbors=6 )  
3 ...
```

Listing 11: Prueba 2 con Imagen 2

Si disminuimos el valor de `scaleFactor` a 1,35 el programa reconocerá todas las caras, como se muestra en la Figura 13. No hay falsos positivos.

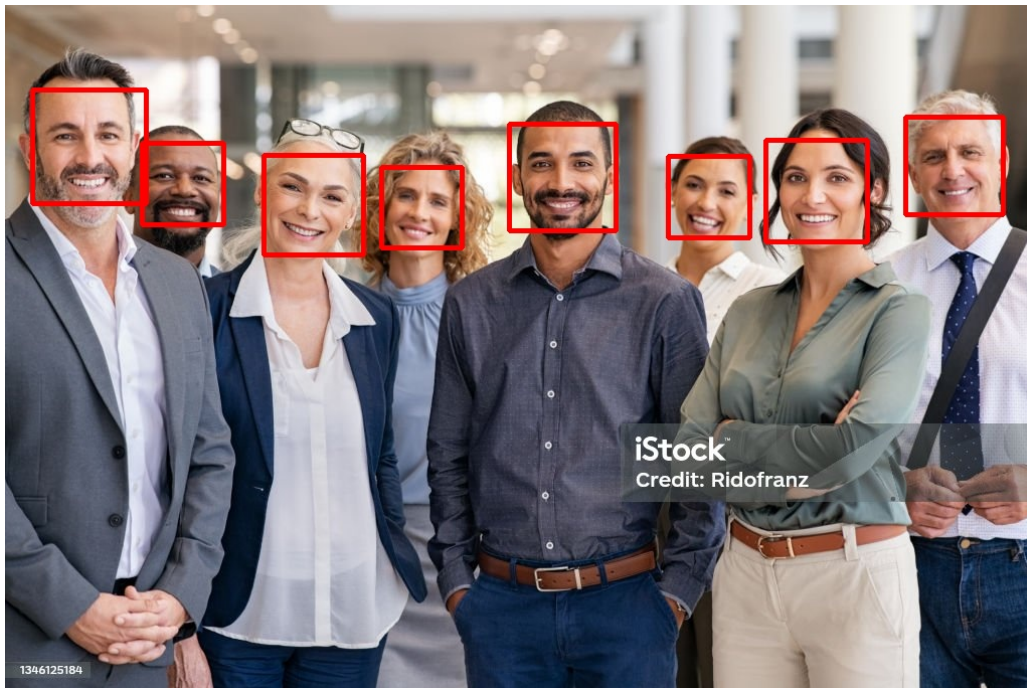


Figura 13: Resultado de *imagen2* con `scaleFactor=1.35`

4.2. Ejercicio 2

Para resolver este ejercicio se necesitan las mismas librerías que en el primer ejercicio (Ver Listing 2). Al igual que en el primer ejercicio, el script empieza comprobando los argumentos de la terminal al ejecutar el programa. Al exigir la sintaxis `python ejercicio2.py <video>`, se hace más dinámica la prueba del programa con distintos videos sin tener que modificar código.

Antes de procesar la señal de vídeo se estructura el archivo de salida. Mediante el método `camara.get()`, el sistema consulta los descriptores de la cabecera del archivo multimedia para extraer tres variables clave: la tasa de refresco (`CAP_PROP_FPS`) y las dimensiones de los cuadros (`WIDTH` y `HEIGHT`). Se añade además una estructura condicional que actúa como salvaguarda fijando un muestreo estándar de 30 FPS en caso de que el archivo de origen posea un *framerate* variable no detectable.

```
1 # Extrae las propiedades nativas del vídeo original
2 fps_leido = int(camara.get(cv2.CAP_PROP_FPS)) # Cuadros por segundo
3 ancho = int(camara.get(cv2.CAP_PROP_FRAME_WIDTH)) # Resolución: Ancho en píxeles
4 alto = int(camara.get(cv2.CAP_PROP_FRAME_HEIGHT)) # Resolución: Alto en píxeles
5
6 # Control de seguridad.
7 fps = int(fps_leido) if fps_leido > 0 else 30
```

Listing 12: Extracción de propiedades del video

A diferencia de trabajar con imágenes, donde no es necesario conocer las propiedades del archivo para usar la función `cv2.imshow()`, para trabajar con `cv2.VideoWriter` se necesitan saber los metadatos (`CAP_PROP_FPS`, `FRAME_WIDTH` y `FRAME_HEIGHT`) para que el archivo final se reproduzca a la misma velocidad que el original.

Para evitar fallos a la hora de almacenar el video resultante en disco no se usan rutas relativas. Con el módulo `os` se determina la ubicación física exacta del script en el disco duro y se aísla el nombre puro del fichero multimedia.

```
1 # Obtiene la ruta absoluta de la carpeta donde se está ejecutando el script
2 directorio_actual = os.path.dirname(os.path.abspath(__file__))
3
4 # Extrae solo el nombre del archivo de vídeo (eliminando rutas previas si las hay)
5 nombre_archivo_puro = os.path.basename(nombre_video)
6 nombre_base, _ = os.path.splitext(nombre_archivo_puro)
7
8 # Combinamos todo en una ruta absoluta limpia para Windows
9 video_salida = os.path.join(directorio_actual, f"resultado_{nombre_base}.avi") #
    Guarda en formato AVI para máxima compatibilidad con códecs en Windows
```

Listing 13: Ruta de guardado

Se usa el códec XVID empaquetado bajo un contenedor binario de extensión `.avi`, asegurando compatibilidad con mi ordenador.

```
1 # Definir el códec de compresión de vídeo.
2 fourcc = cv2.VideoWriter_fourcc(*'XVID')
3
4 # Instanciar el objeto grabador con la ruta, el códec, los FPS y el tamaño de los
    fotogramas
5 grabador = cv2.VideoWriter(video_salida, fourcc, fps, (ancho, alto), isColor=True)
6
7 # Validar que el grabador se haya inicializado correctamente en el sistema operativo
8 if not grabador.isOpened():
9     print("[ERROR] No se pudo inicializar el grabador de vídeo. Revisa los permisos
    de la carpeta.")
10     sys.exit(1)
```

Listing 14: Códec y extensión

Dentro del bucle principal de ejecución, el proceso se divide en tres fases por cada *frame* capturado:

- Se transforma el plano de color empleando `cv2.COLOR_BGR2GRAY` como se hacía en el tratamiento de imágenes.
- Se establecen los valores de `scaleFactor` y `minNeighbors` para la función `detectMultiScale`.

- Se pintan los rectángulos sobre la matriz original a color (`img`) para no degradar la calidad estética del vídeo final. Acto seguido, se efectúa un volcado físico directo al disco mediante `grabador.write()` y, de forma paralela, se proyecta la imagen en la interfaz gráfica del usuario (`cv2.imshow()`).

```

1 #Procesamiento frame a frame del vídeo
2 while camara.isOpened():
3     hay_frame, img = camara.read()
4
5     # Si 'hay_frame' es False, significa que el vídeo ha llegado a su fin
6     if not hay_frame:
7         print("Fin del archivo de vídeo alcanzado.")
8         break
9
10    # Convertir el frame a escala de grises para el algoritmo de Viola-Jones
11    gris = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
12
13    # Detectar rostros en el fotograma actual
14    faces = modeloCara.detectMultiScale(gris, scaleFactor=1.3, minNeighbors=4)
15
16    # Iterar sobre cada rostro detectado en el fotograma actual
17    for (x, y, w, h) in faces:
18        # Dibujamos el rectángulo delimitador sobre el frame original a color (img)
19        # Color: Azul (255, 0, 0) en formato BGR.
20        # Grosor de línea: 3 píxeles
21        cv2.rectangle(img, (x, y), (x + w, y + h), (255, 0, 0), 3)
22
23    # Escribir el fotograma modificado (con rectángulos ya pintados) en el archivo de
24    # salida
25    grabador.write(img)
26
27    # Desplegar el frame en tiempo real en una ventana interactiva de Windows
28    cv2.imshow('Ejercicio 2 - Detección Facial en Vídeo', img)

```

Listing 15: Frame a frame en ejercicio2

Con la función `waitKey` se captura si hay una pulsación de tecla. Si la tecla pulsada es la que corresponde con el número 27 en ASCII, es decir, la tecla `Escape` (`Esc`), se fuerza la salida del bucle.

```

1 while camara.isOpened():
2     ...
3     # Escuchar los eventos del teclado durante 1 milisegundo
4     key = cv2.waitKey(1)
5     # 27 es el código ASCII para la tecla 'Esc' (Escape)
6     if key == 27:
7         print("Proceso interrumpido manualmente por el usuario.")
8         break

```

Listing 16: Cerrar ventana

Una vez que se ha salido del bucle `while` se realiza el vaciado de búfer.

```

1 # Liberar los descriptores para que Windows cierre correctamente los archivos de
2 # video
3 camara.release()
4 grabador.release() # Al liberar el grabador se finaliza la escritura física del
5 # archivo nuevo
6 cv2.destroyAllWindows()
7 print(f"[ÉXITO] Archivo de vídeo exportado correctamente en la raíz del proyecto.\n")

```

Listing 17: Limpieza del búfer

4.2.1. Prueba con *Video 2*

Orden necesaria para la terminal:

```
(venv) PS C:\Users\inesr\PDIH\P6> python .\ejercicio2.py .\v2.mp4
```

Listing 18: Orden de ejecución - Video 2

```
1 ...  
2     faces = modeloCara.detectMultiScale(gris, scaleFactor=1.3, minNeighbors=4)  
3 ...
```

Listing 19: Prueba 1 con Video 2

Con la siguiente llamada a la función `detectMultiScale(gris, scaleFactor=1.3, minNeighbors=4)` se capturan las caras de las dos personas aunque puede detectar falsos positivos. En la Figura 14 se puede ver como se recuadran los dos rostros detectados.

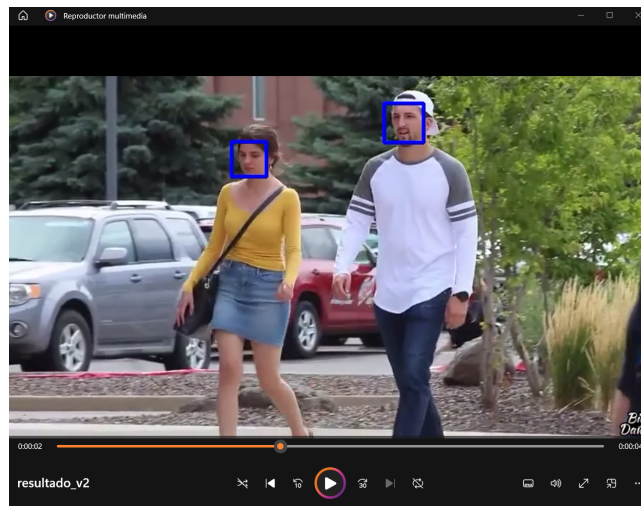


Figura 14: Captura del video 2

En la siguiente dirección se puede ver el vídeo completo con los rostros recuadrados: https://github.com/ruiz314/PDIH/blob/main/P6/ejer2/resultado_v2.avi.

4.2.2. Prueba con Video 3

Orden necesaria para la terminal:

```
(venv) PS C:\Users\inesr\PDIH\P6> python .\ejercicio2.py .\v3.mp4
```

Listing 20: Orden de ejecución - Video 3

```
1 ...  
2     faces = modeloCara.detectMultiScale(gris, scaleFactor=1.3, minNeighbors=4)  
3 ...
```

Listing 21: Prueba 1 con Video 3

Con la siguiente llamada a la función `detectMultiScale(gris, scaleFactor=1.3, minNeighbors=4)` se captura perfectamente la cara de la persona que está hablando. En la Figura 15 se puede ver como se recuadra el rostro detectado.

En la siguiente dirección se puede ver el vídeo completo con el rostro recuadrado: https://github.com/ruiz314/PDIH/blob/main/P6/ejer2/resultado_v3.avi.



Figura 15: Captura del video 3

4.3. Ejercicio 3

4.3.1. Ejemplo de gatos

El código fuente proporcionado por el profesor se encuentra en el Anexo 5.6. El material necesario (imagen y fichero *.xml*) están en la siguiente dirección del repositorio: <https://github.com/ruiz314/PDIH/blob/main/P6/ejemplos/gatos/ejemplo-gatos.py>.

En el ejemplo se usa el clasificador *haarcascade_frontalcatface_extended.xml* y se usa la función *detectMultiScale*:

```
1 ...
2 gatos = modelo.detectMultiScale(imagen, scaleFactor=1.1, minNeighbors=6)
3 ...
```

Listing 22: Ejemplo de gatos

La imagen inicial es la que se muestra en la Figura 16. Muestra tres gatos de distinto tamaño y color.



Figura 16: Imagen inicial de gatos

Tras ejecutar el programa observamos que se reconocen los tres gatos en la Figura 17.

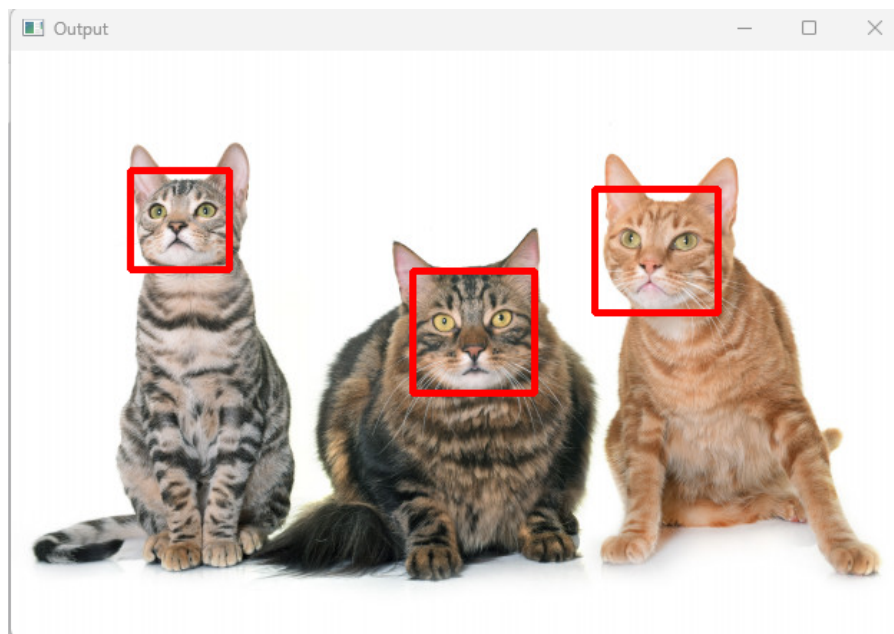


Figura 17: Imagen con gatos reconocidos

4.3.2. Resolución de ejercicio 3

Los videos han sido descargados en la página *Pixabay*:

- Video 1 - Gato saltando de una caja de cartón: https://www.istockphoto.com/es/v%C3%ADdeo/problema-de-alimentaci%C3%B3n-en-animales-gm2158898087-579506401?utm_source=pixabay&utm_medium=affiliate&utm_campaign=sponsored_video&utm_content=srp_topbanner_media&utm_term=gatos.
- Video 2 - Gato andando por el césped: <https://pixabay.com/es/videos/gato-gato-naranja-gato-domestico-179212/>.

En mi repositorio se encuentran en la siguiente ruta: <https://github.com/ruiz314/PDIH/tree/main/P6/ejer3>.

Se ha usado el clasificador `haarcascade_frontalcatface_extended.xml`. Al igual que se hace en los otros ejercicios, se usan las librerías `sys` y `os` (*Listing 2*).

El script tiene la misma estructura que el programa del ejercicio 2 (Sección 4.2):

- Comprueba que los argumentos pasados desde la terminal sean correctos.
- La extracción de las propiedades del video pasado como argumento se hace igual que para el ejercicio 2 (*Listing 12*).
- El video resultante con el gato detectado se almacena en el disco (ver *Listing 13* y *Listing 14*).
- El bucle principal de ejecución sigue la misma estructura que el programa de reconocimiento de caras de personas (ver *Listing 15* y *16*).

Los rectángulos delimitadores se han configurado en color rojo para diferenciarlos del ejercicio de rostros humanos.

4.3.3. Prueba con Video 1

Orden necesaria para la terminal:

```
(venv) PS C:\Users\inesr\PDIH\P6> python .\ejercicio3.py .\video1.mp4
```

Listing 23: Orden de ejecución - Video 1


```

1 ...
2     gatos = modeloGato.detectMultiScale(imagen_gris, scaleFactor=1.1, minNeighbors=6)
3 ...

```

Listing 24: Prueba 1 con Video 1

Con la siguiente llamada a la función `detectMultiScale` (Listing 24) se captura perfectamente el gato que salta de la caja de cartón, pero también aparecen algunos falsos positivos. En la Figura 18 se puede ver como se recuadra el gato detectado.

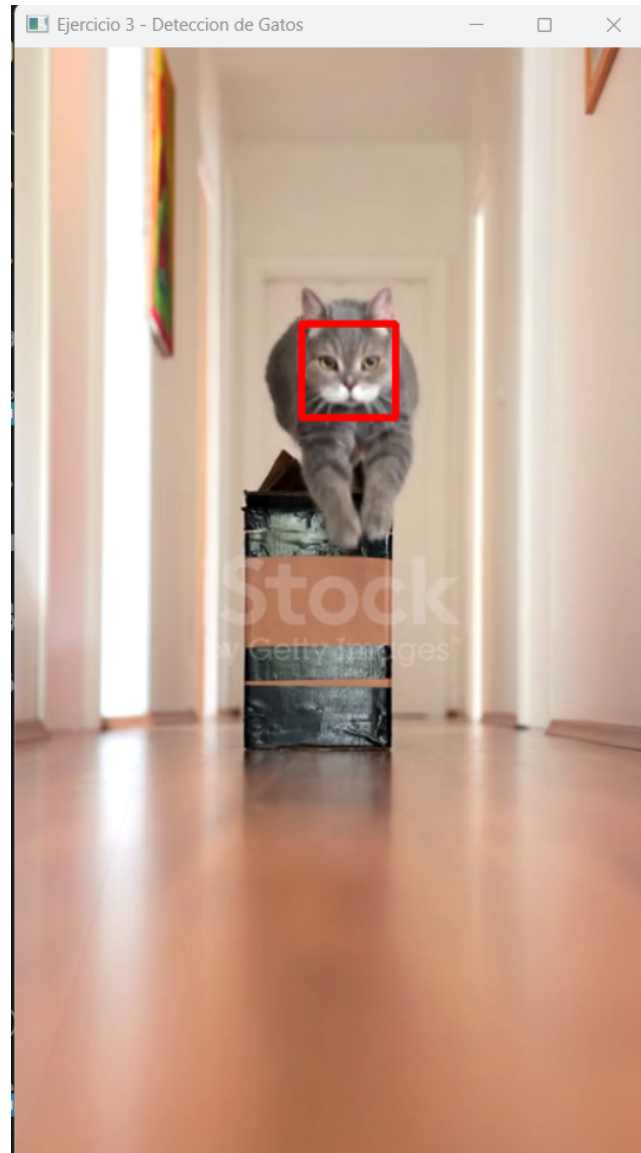


Figura 18: Captura del vídeo 1

En la siguiente dirección se puede ver el vídeo completo con el gato recuadrado: https://github.com/ruiz314/PDIH/blob/main/P6/ejer3/resultado_gatos_video1.avi.

4.3.4. Prueba con Video 2

Orden necesaria para la terminal:

```
(venv) PS C:\Users\inesr\PDIH\P6> python .\ejercicio3.py .\video2.mp4
```

Listing 25: Orden de ejecución - Video 2

Los distintos videos obtenidos tras cambiar el factor están en la carpeta: https://drive.google.com/drive/folders/1ivjwcP9SK6ACY38NjBbvBy6EhtWFq_ly?usp=drive_link

```

1 ...
2     gatos = modeloGato.detectMultiScale(imagen_gris, scaleFactor=1.1, minNeighbors=6)
3 ...

```

Listing 26: Prueba 2 con Video 2

Con la siguiente llamada a la función `detectMultiScale()` (Listing 26) se captura perfectamente el gato que anda por el césped, aunque también detecta falsos positivos. En la Figura 19 se puede ver como se recuadra el gato detectado.

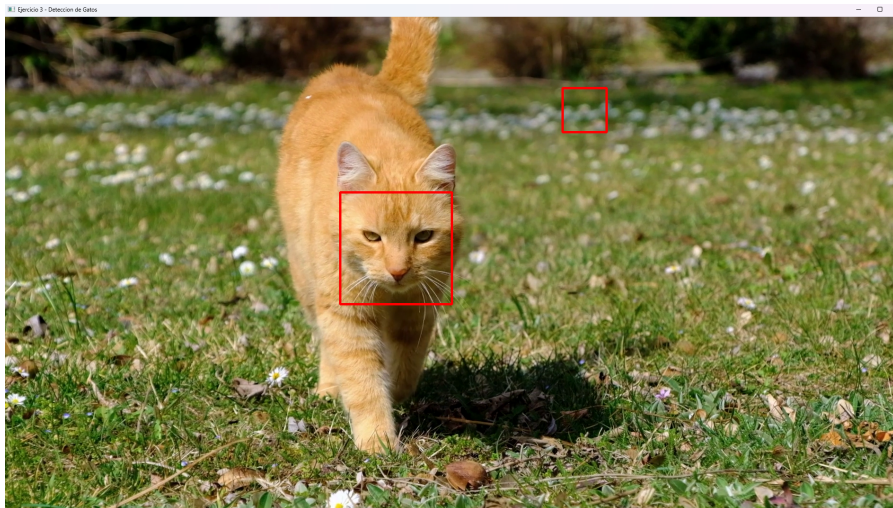


Figura 19: Captura del vídeo 2

En la siguiente dirección se puede ver el vídeo completo con el gato recuadrado: https://drive.google.com/file/d/1q0p9S4KGT94DBaVb8YSx2Lgk-Z45CJVB/view?usp=drive_link.

Si se sube el factor a 1,5 el gato no será detectado, pero con 1,4 se detectará en algunas ocasiones. En la siguiente dirección se puede ver el vídeo con `scaleFactor=1.4`: https://drive.google.com/file/d/1yiVEo32w5bx9WRZapWZwXa0iCzrHiu9Z/view?usp=drive_link.

4.4. Ejercicio 4

4.4.1. Detección en imagen

Se han usado los siguientes clasificadores:

- `haarcascade_frontalface_default.xml`
- `haarcascade_fullbody.xml`

Se usan las mismas librerías que en los otros ejercicios (ver Listing 2).

El programa sigue la siguiente estructura:

- Comprueba que los argumentos pasados desde la terminal sean correctos.
- Carga los modelos que se usan.
- Carga la imagen y la pasa a tonos de grises.
- Busca las caras y los cuerpos con la función `detectMultiScale`.
- Dibuja los rectángulos con `cv2.rectangle`.

Es importante hacer dos llamadas a la función `detectMultiScale`, una vez para los rostros y otra para los cuerpos.

```

1 # Procesamiento en escala de grises
2 imagen = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
3

```

```

4 # Buscar los rostros:
5 rostros = modeloCara.detectMultiScale(imagen, scaleFactor=1.5, minNeighbors=6)
6
7 # Buscar los cuerpos:
8 cuerpos = modeloCuerpo.detectMultiScale(imagen, scaleFactor=1.5, minNeighbors=6)
9
10 # Dibujar rectángulos
11 for (x,y,ancho, alto) in rostros:
12     cv2.rectangle(img, (x,y), (x+ancho, y+alto), (0,0,255) , 3)
13 for (x,y,ancho, alto) in cuerpos:
14     cv2.rectangle(img, (x,y), (x+ancho, y+alto), (255,0,0) , 3)

```

Listing 27: Búsqueda de caras y cuerpos

4.4.2. Prueba con imagen

La imagen de prueba es la proporcionada por el profesor, y se puede ver en Figura 20.



Figura 20: Imagen de prueba ejercicio4

Para ejecutar se necesita la siguiente orden:

```
(venv) PS C:\Users\inesr\PDIH\P6> python ejercicio4_img.py imagen_input.jpg
```

Listing 28: Orden de ejecución para detectar cara y cuerpo en imagen

La primera prueba que detecta alguna cara y algún cuerpo es con los siguiente valores para los factores:

```

1 # Buscar los rostros:
2 rostros = modeloCara.detectMultiScale(imagen, scaleFactor=1.5, minNeighbors=6)
3
4 # Buscar los cuerpos:
5 cuerpos = modeloCuerpo.detectMultiScale(imagen, scaleFactor=1.3, minNeighbors=3)

```

Listing 29: Prueba 1 con imagen para ejercicio 4

El resultado obtenido se puede ver en la Figura 21. Se puede observar que no se reconoce el cuerpo del modelo principal, pero sí se reconoce un cuerpo de una persona que está en segundo plano en la imagen.

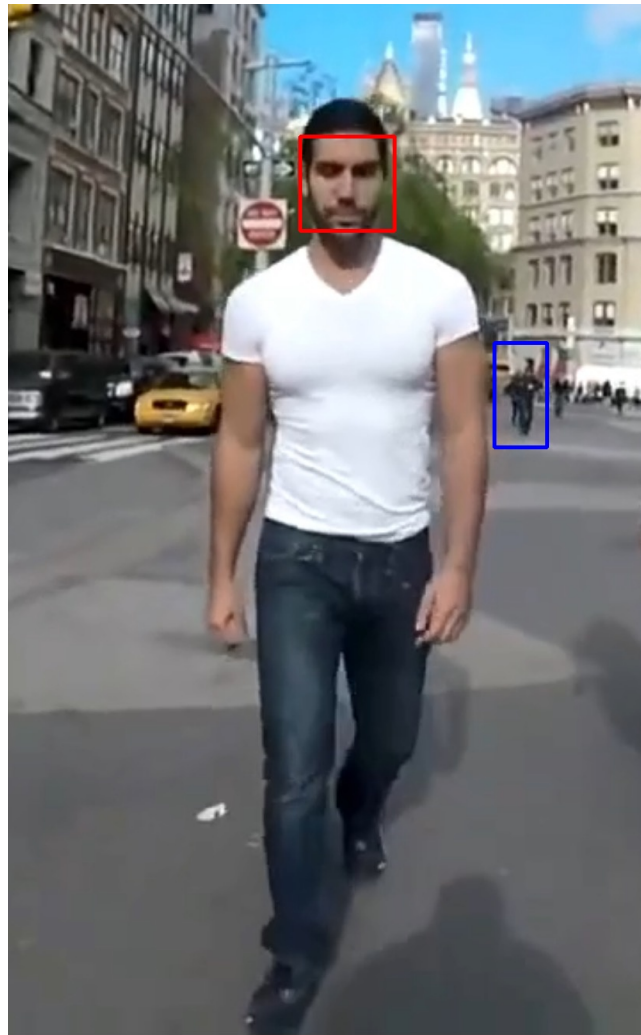


Figura 21: Resultado ejercicio 4 con imagen

Si reducimos el valor `scaleFactor` en ambos modelos conseguiremos que se reconozca otro cuerpo en el fondo, pero no el modelo principal. También detecta una cara en las arrugas del pantalón del modelo. Ver Figura 23.

```

1 rostros = modeloCara.detectMultiScale(imagen, scaleFactor=1.3, minNeighbors=6)
2 cuerpos = modeloCuerpo.detectMultiScale(imagen, scaleFactor=1.1, minNeighbors=3)

```

Listing 30: Prueba 2 con imagen para ejercicio 4

En una tercera prueba, buscando reconocer el cuerpo del modelo principal, he bajado el factor de escala del modelo cuerpo y se ha conseguido detectar al modelo. Ver Figura 23.

```

1 rostros = modeloCara.detectMultiScale(imagen, scaleFactor=1.5, minNeighbors=6)
2 cuerpos = modeloCuerpo.detectMultiScale(imagen, scaleFactor=1.01, minNeighbors=3)

```

Listing 31: Prueba 2 con imagen para ejercicio 4

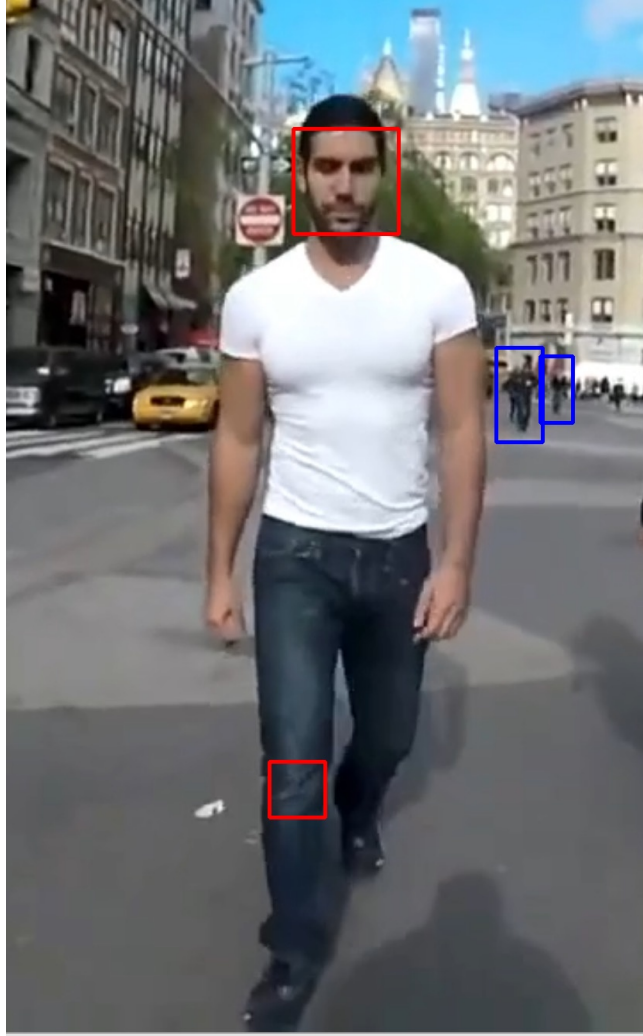


Figura 22: Resultado ejercicio 4 con imagen (prueba 2)

Al tener un valor más alto para el factor de escala en el modelo de cara no se detecta ninguna falsa cara en el pantalón del modelo.

4.4.3. Detección en video

El programa sigue la siguiente estructura:

- Comprueba que los argumentos pasados desde la terminal sean correctos.
- Carga los modelos que se usan.
- Inicializa la captura del archivo de video
- Extrae las propiedades del video para poder exportar el video resultante con la misma escala.
- Por cada fotograma:
 - Convierte el fotograma en escala de grises
 - Busca las caras y los cuerpos con la función `detectMultiScale`.
 - Dibuja los rectángulos con `cv2.rectangle`.
- Se exporta el video y se muestra por pantalla.

Como en los ejercicios previos, así se trabaja con un video:

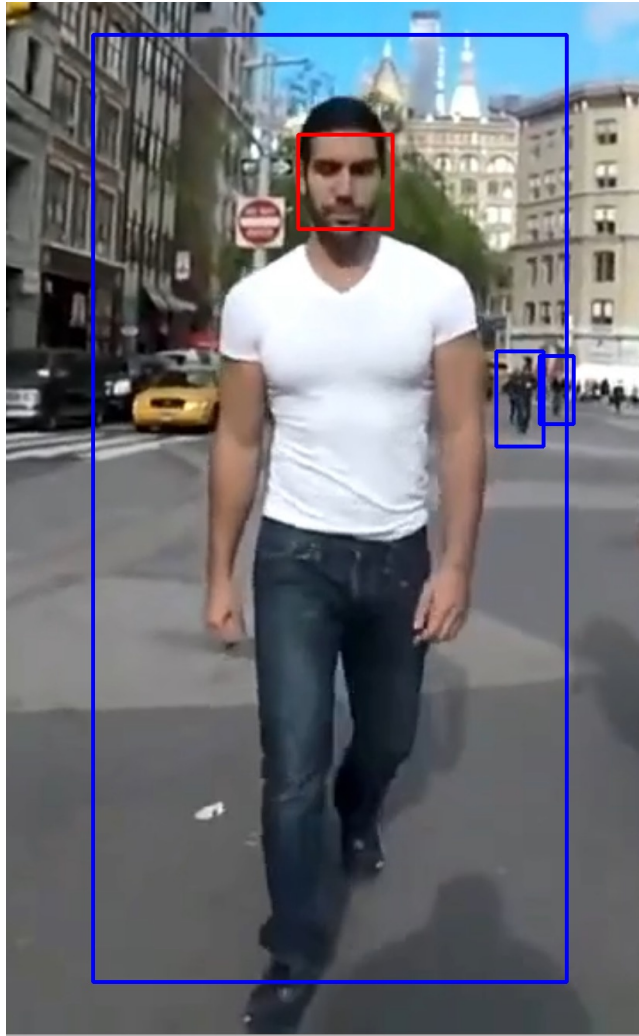


Figura 23: Resultado ejercicio 4 con imagen (prueba 3)

```

1 # Procesamiento frame a frame del vídeo
2 while camara.isOpened():
3     hay_frame, img = camara.read()
4
5     if not hay_frame:
6         print("Fin del archivo de vídeo alcanzado.")
7         break
8
9     # Convertimos el frame a escala de grises para el algoritmo de Viola-Jones
10    gris = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
11
12    # Buscar los rostros:
13    rostros = modeloCara.detectMultiScale(gris, scaleFactor=1.5, minNeighbors=6)
14
15    # Buscar los cuerpos:
16    cuerpos = modeloCuerpo.detectMultiScale(gris, scaleFactor=1.5, minNeighbors=3)
17
18    # Dibujar rectángulos
19    for (x,y,ancho, alto) in rostros:
20        cv2.rectangle(img, (x,y), (x+ancho, y+alto), (0,0,255), 3) # Rojo
21    for (x,y,ancho, alto) in cuerpos:
22        cv2.rectangle(img, (x,y), (x+ancho, y+alto), (255,0,0), 3) # Azul
23    ...

```

Listing 32: Procesamiento frame a frame de video para ejercicio 4

4.4.4. Prueba con Video 4

Con los siguientes parámetros se consigue detectar en casi todo momento la cara del modelo. Hay algunos falsos positivos. También se llegan a detectar cuerpos.

```
1 # Buscar los rostros:
2 rostros = modeloCara.detectMultiScale(gris, scaleFactor=1.5, minNeighbors=6)
3
4 # Buscar los cuerpos:
5 cuerpos = modeloCuerpo.detectMultiScale(gris, scaleFactor=1.5, minNeighbors=3)
```

Listing 33: Prueba 1 con video4 para ejercicio 4

En un trozo del video se pixela a una persona, y el programa reconoce el cuerpo de cintura para abajo.

El video resultante se puede ver en mi directorio de GitHub: https://github.com/ruiz314/PDIH/blob/main/P6/ejer4/resultado_v4.avi

Si se reduce `scaleFactor` a 1,3 en el clasificador del cuerpo, en la primera parte del video no se detecta el cuerpo del modelo principal, pero en la segunda parte si se detecta.

4.4.5. Prueba con Video 5

Con los siguientes parámetros se consigue detectar en casi todo momento la cara del modelo. Hay algunos falsos positivos y algunas caras que se detectan dos veces. También se llegan a detectar cuerpos.

```
1 # Buscar los rostros:
2 rostros = modeloCara.detectMultiScale(gris, scaleFactor=1.5, minNeighbors=6)
3
4 # Buscar los cuerpos:
5 cuerpos = modeloCuerpo.detectMultiScale(gris, scaleFactor=1.5, minNeighbors=3)
```

Listing 34: Prueba 1 con video4 para ejercicio 4

El video resultante se puede ver en mi directorio de GitHub: https://github.com/ruiz314/PDIH/blob/main/P6/ejer4/resultado_v5.avi

5. Anexo

5.1. Fichero *ejemplo-fotos-1.py*

Ejemplo proporcionado por el profesor. En el repositorio de *GitHub* está en la siguiente ruta: <https://github.com/ruiz314/PDIH/blob/main/P6/ejemplos/caras/ejemplo-fotos-1.py>.

```
1 # ejemplo-fotos-1.py
2 import cv2
3
4 #Cargamos nuestro modelo classificador de Haar:
5 modelo = cv2.CascadeClassifier('haarcascade_frontalface_alt.xml')
6
7 #Cargamos la imagen
8 img = cv2.imread('imagen_input.jpg')
9 imagen = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
10
11 #Buscamos los rostros:
12 rostros = modelo.detectMultiScale(imagen, scaleFactor=1.5, minNeighbors=6 )
13 # scaleFactor y minNeighbors son parámetros estándar para esta función.
14 # El primero es el factor de escala: la función intentará encontrar rostros
15 # escalando la imagen varias veces, y este factor indica en cuánto se reduce la
16 # imagen cada vez.
17 # El segundo parámetro indica la calidad de las detecciones: un valor elevado resulta
18 # en menos detecciones pero con más fiabilidad.
19 # Si hacemos scaleFactor más cercano a 1.0 tardará un poco más de tiempo, pero será m
20 # ás fiable (no se le escaparán algunas caras)
21
22 #Ahora recorremos el array 'rostros' y dibujamos los rectángulos sobre la imagen
23 # original:
24 for (x,y,ancho, alto) in rostros:
25     cv2.rectangle(img, (x,y), (x+ancho, y+alto), (0,0,255) , 3)
26
27 #Abrimos una ventana con el resultado:
28 cv2.imshow('Output', img)
29 print("\nMostrando resultado. Pulsa cualquier tecla para salir.\n")
30 cv2.waitKey(0)
31 cv2.destroyAllWindows()
```

Listing 35: Código completo del ejemplo1

5.2. Fichero *ejercicio1.py*

```
1 import cv2
2 import sys
3 import os
4
5 # Validación del número de argumentos en la línea de comandos
6 if len(sys.argv) < 2:
7     print("[ERROR] Uso correcto: python ejercicio1.py <nombre_imagen.jpg>")
8     sys.exit(1)
9
10 # Captura del parámetro del archivo de imagen
11 nombre_imagen = sys.argv[1]
12
13 #Cargamos nuestro modelo classificador de Haar:
14 modelo = cv2.CascadeClassifier('haarcascade_frontalface_alt.xml')
15
16 #Cargamos la imagen
17 img = cv2.imread(nombre_imagen)
18
19 # Control de excepciones ante rutas inexistentes o corruptas
20 if img is None:
```

```

21     print(f"[ERROR] No se pudo cargar la imagen '{nombre_imagen}')"
22     sys.exit(1)
23
24 # Procesamiento en escala de grises
25 imagen = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
26
27 #Buscamos los rostros:
28 rostros = modelo.detectMultiScale(imagen, scaleFactor=1.35, minNeighbors=6 )
29
30 #Ahora recorremos el array 'rostros' y dibujamos los rectángulos sobre la imagen
    original:
31 for (x,y,ancho, alto) in rostros:
32     cv2.rectangle(img, (x,y), (x+ancho, y+alto), (0,0,255) , 3)
33
34 #Imprimir el número de rostros detectados
35 print(f"Número de rostros detectados con éxito: {len(rostros)}")
36
37 #Guardar la imagen en disco
38 # Separamos el nombre del archivo de su extensión (ej: 'foto' y '.jpg')
39 nombre_base, extension = os.path.splitext(nombre_imagen)
40 # Creamos el nuevo nombre para el archivo de salida
41 nombre_salida = f"resultado_{nombre_base}{extension}"
42
43 # Guardamos la imagen modificada en la carpeta de trabajo
44 cv2.imwrite(nombre_salida, img)
45 print(f"Imagen resultante guardada con éxito como: {nombre_salida}")
46
47 #Abrimos una ventana con el resultado:
48 cv2.imshow(f'Imagen: - {nombre_imagen}', img)
49 print("\nMostrando resultado del ejercicio 1. Pulsa cualquier tecla para salir.\n")
50 cv2.waitKey(0)
51 cv2.destroyAllWindows()

```

Listing 36: Código completo del ejercicio1

5.3. Fichero *ejemplo2.py*

Ejemplo proporcionado por el profesor. En el repositorio de *GitHub* está en la siguiente ruta: <https://github.com/ruiz314/PDIH/blob/main/P6/ejemplos/en-videos/ejemplo2.py>.

```

1 import cv2
2
3 # Cargamos el modelo clasificador para la cara
4 modeloCara = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
5
6 # Abrimos la cámara o cargamos un vídeo:
7 #camara = cv2.VideoCapture(-1)
8 camara = cv2.VideoCapture('v2.mp4')
9
10 while camara.isOpened():
11     # Cargamos en img lo capturado por la cámara
12     _, img = camara.read()
13
14     color = cv2.cvtColor(img, cv2.COLOR_BGR2BGRa)
15
16     # Detectamos la cara
17     faces = modeloCara.detectMultiScale(color, 1.1, 4)
18
19     # Pintamos el rectángulo sobre la cara
20     for (x, y , w ,h) in faces:
21         cv2.rectangle(img, (x,y), (x+w, y+h), (255, 0 , 0), 3)
22
23     # Lo mostramos en pantalla mientras no pulsemos la tecla 'esc'
24     cv2.imshow('img', img)
25     key = cv2.waitKey(1)

```

```

26     if key == 27:
27         break
28
29 camara.release()
30 cv2.destroyAllWindows()

```

Listing 37: Código completo del ejemplo2

5.4. Fichero *ejemplo3.py*

Ejemplo proporcionado por el profesor. En el repositorio de *GitHub* está en la siguiente ruta: <https://github.com/ruiz314/PDIH/blob/main/P6/ejemplos/en-videos/ejemplo3.py>.

```

1 import cv2
2
3 # Cargamos clasificadores para cara y ojos
4 modeloCara = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
5 modeloOjo = cv2.CascadeClassifier('haarcascade_eye_tree_eyeglasses.xml')
6
7 # abrimos el flujo de vídeo desde la cámara o desde un vídeo
8 #camara = cv2.VideoCapture(-1)
9 camara = cv2.VideoCapture('v3.mp4')
10
11 while camara.isOpened():
12     _, img = camara.read()
13     color = cv2.cvtColor(img, cv2.COLOR_BGR2BGRA)
14
15     faces = modeloCara.detectMultiScale(color, 1.3, 4)
16     for (x, y, w, h) in faces:
17         # Dibujamos rectángulo en las caras detectadas
18         cv2.rectangle(img, (x,y), (x + w, y + h), (255, 0, 0), 3)
19
20     eyes = modeloOjo.detectMultiScale(color, 1.3, 4)
21     for (x, y, w, h) in eyes:
22         # Dibujamos rectángulo en los ojos
23         cv2.rectangle(img, (x,y), (x + w, y + h), (0, 255, 0), 3)
24
25     cv2.imshow('img', img)
26     key = cv2.waitKey(1)
27     if key == 27:
28         break
29
30 camara.release()
31 cv2.destroyAllWindows()

```

Listing 38: Código completo del ejemplo3

5.5. Fichero *ejercicio2.py*

```

1 import cv2
2 import sys
3 import os
4
5 # Validación del número de argumentos en la línea de comandos
6 if len(sys.argv) < 2:
7     print("\n[ERROR] Uso correcto: python ejercicio2.py <nombre_video.mp4>\n")
8     sys.exit(1)
9
10 # Captura del parámetro del archivo de video
11 nombre_video = sys.argv[1]
12
13 #Carga el modelo classificador de Haar:
14 modeloCara = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')

```



```

15
16 # Inicializa la captura del archivo de vídeo pasado por parámetro
17 camara = cv2.VideoCapture(nombre_video)
18
19 # Valida que el archivo realmente exista y OpenCV pueda decodificarlo
20 if not camara.isOpened():
21     print(f"\n[ERROR] No se pudo abrir el archivo de vídeo: '{nombre_video}'\n")
22     sys.exit(1)
23
24 # Extrae las propiedades nativas del vídeo original para que el de salida sea idé
    ntico
25 fps_leido = int(camara.get(cv2.CAP_PROP_FPS)) # Cuadros por segundo
26 ancho = int(camara.get(cv2.CAP_PROP_FRAME_WIDTH)) # Resolución: Ancho en píxeles
27 alto = int(camara.get(cv2.CAP_PROP_FRAME_HEIGHT)) # Resolución: Alto en píxeles
28
29 # Control de seguridad: Si los FPS no se leen bien, asignamos 30 por defecto
30 fps = int(fps_leido) if fps_leido > 0 else 30
31
32 # Guardar video de salida
33 # Obtiene la ruta absoluta de la carpeta donde se está ejecutando el script
34 directorio_actual = os.path.dirname(os.path.abspath(__file__))
35
36 # Extrae solo el nombre del archivo de vídeo (eliminando rutas previas si las hay)
37 nombre_archivo_puro = os.path.basename(nombre_video)
38 nombre_base, _ = os.path.splitext(nombre_archivo_puro)
39
40 # Combinar todo en una ruta absoluta limpia para Windows
41 video_salida = os.path.join(directorio_actual, f"resultado_{nombre_base}.avi")
42
43 # Define el códec de compresión de vídeo.
44 fourcc = cv2.VideoWriter_fourcc(*'XVID')
45
46 # Instancia el objeto grabador
47 # Parámetros: ruta, códec, FPS y tamaño de los fotogramas
48 grabador = cv2.VideoWriter(video_salida, fourcc, fps, (ancho, alto), isColor=True)
49
50 # Validación de que el grabador se haya inicializado correctamente
51 if not grabador.isOpened():
52     print("[ERROR] No se pudo inicializar el grabador de vídeo. Revisa los permisos
    de la carpeta.")
53     sys.exit(1)
54
55 print(f"\nProcesando vídeo: '{nombre_video}' ({ancho}x{alto} a {fps} FPS)")
56 print(f"El resultado con las detecciones se guardará en: '{video_salida}'")
57 print("Presiona la tecla 'ESC' en la ventana gráfica para interrumpir el proceso.\n")
58
59 #Procesamiento frame a frame del vídeo
60 while camara.isOpened():
61     hay_frame, img = camara.read()
62
63     # Si 'hay_frame' es False, significa que el vídeo ha llegado a su fin
64     if not hay_frame:
65         print("Fin del archivo de vídeo alcanzado.")
66         break
67
68     # Convierte el frame a escala de grises para el algoritmo de Viola-Jones
69     gris = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
70
71     # Detecta rostros en el fotograma actual
72     faces = modeloCara.detectMultiScale(gris, scaleFactor=1.3, minNeighbors=4)
73
74     # Itera sobre cada rostro detectado en el fotograma actual
75     for (x, y, w, h) in faces:

```

```

76         # Dibuja el rectángulo delimitador sobre el frame original a color (img)
77         # Color: Azul (255, 0, 0) en formato BGR.
78         # Grosor de línea: 3 píxeles
79         cv2.rectangle(img, (x, y), (x + w, y + h), (255, 0, 0), 3)
80
81         # Escribe el fotograma modificado (con rectángulos ya pintados) en el archivo de
82         # salida
83         grabador.write(img)
84
85         # Despliega el frame en tiempo real en una ventana interactiva de Windows
86         cv2.imshow('Ejercicio 2 - Detección Facial en Vídeo', img)
87
88         # Escucha los eventos del teclado durante 1 milisegundo
89         key = cv2.waitKey(1)
90         # 27 es el código ASCII para la tecla 'Esc' (Escape)
91         if key == 27:
92             print("Proceso interrumpido manualmente por el usuario.")
93             break
94
95     # Liberar los descriptores para que Windows cierre correctamente los archivos de
96     # video
97     camara.release()
98     grabador.release() # Al liberar el grabador se finaliza la escritura física del
99     # archivo nuevo
100    cv2.destroyAllWindows()
101    print(f"Archivo de vídeo exportado correctamente en la raíz del proyecto.\n")

```

Listing 39: Código completo del ejercicio2

5.6. Fichero *ejemplo-gatos.py*

Ejemplo proporcionado por el profesor. En el repositorio de *GitHub* está en la siguiente ruta: <https://github.com/ruiz314/PDIH/blob/main/P6/ejemplos/gatos/ejemplo-gatos.py>.

```

1  import cv2
2
3  modelo = cv2.CascadeClassifier('haarcascade_frontalcatface_extended.xml')
4
5  img = cv2.imread('imagen_input.jpg')
6  imagen = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
7
8  gatos = modelo.detectMultiScale(imagen, scaleFactor=1.1, minNeighbors=6 )
9
10 for (x,y,ancho, alto) in gatos:
11     cv2.rectangle(img, (x,y), (x+ancho, y+alto), (0,0,255) , 3)
12
13 cv2.imshow('Output', img)
14 print("\nMostrando resultado. Pulsa cualquier tecla para salir.\n")
15 cv2.waitKey(0)
16 cv2.destroyAllWindows()

```

Listing 40: Código completo del ejemplo-gatos

5.7. Fichero *ejercicio3.py*

```

1  import cv2
2  import sys
3  import os
4
5  # Validación del número de argumentos en la línea de comandos
6  if len(sys.argv) < 2:
7      print("\n[ERROR] Uso correcto: python ejercicio3.py <nombre_video.mp4>\n")
8      sys.exit(1)
9

```

```

10 # Captura del parámetro del archivo de video
11 nombre_video = sys.argv[1]
12
13 # Carga el modelo classificador de Haar:
14 modeloGato = cv2.CascadeClassifier('haarcascade_frontalcatface_extended.xml')
15
16 # Inicializa la captura del archivo de video pasado por parámetro
17 camara = cv2.VideoCapture(nombre_video)
18
19 # Valida que el archivo realmente exista
20 if not camara.isOpened():
21     print(f"\n[ERROR] No se pudo abrir el archivo de video: '{nombre_video}'\n")
22     sys.exit(1)
23
24 # Extrae las propiedades nativas del video original
25 fps_leido = int(camara.get(cv2.CAP_PROP_FPS))
26 ancho = int(camara.get(cv2.CAP_PROP_FRAME_WIDTH))
27 alto = int(camara.get(cv2.CAP_PROP_FRAME_HEIGHT))
28
29 # Control de seguridad: Si los FPS no se leen bien, asignamos 30 por defecto para
    evitar el fallo
30 fps = int(fps_leido) if fps_leido > 0 else 30
31
32 # Guardar video de salida
33 directorio_actual = os.path.dirname(os.path.abspath(__file__))
34
35 # Extraer solo el nombre del archivo de video
36 nombre_archivo_puro = os.path.basename(nombre_video)
37 nombre_base, _ = os.path.splitext(nombre_archivo_puro)
38
39 # Combina todo en una ruta absoluta
40 video_salida = os.path.join(directorio_actual, f"resultado_gatos_{nombre_base}.avi")
41
42 # Definir el códec de compresión de video.
43 fourcc = cv2.VideoWriter_fourcc(*'XVID')
44
45 # Instanciar el objeto grabador con la ruta, el códec, los FPS y el tamaño de los
    fotogramas
46 grabador = cv2.VideoWriter(video_salida, fourcc, fps, (ancho, alto), isColor=True)
47
48 # Validación de que el grabador se haya inicializado correctamente en el sistema
    operativo
49 if not grabador.isOpened():
50     print("[ERROR] No se pudo inicializar el grabador de video. Revisa los permisos
    de la carpeta.")
51     sys.exit(1)
52
53 print(f"\n Procesando video de GATOS: '{nombre_video}' ({ancho}x{alto} a {fps} FPS)")
54 print(f"El resultado con las detecciones se guardará en: '{video_salida}'")
55 print("Presiona la tecla 'ESC' en la ventana gráfica para interrumpir el proceso.\n")
56
57 #Procesamiento frame a frame del video
58 while camara.isOpened():
59     hay_frame, img = camara.read()
60
61     if not hay_frame:
62         print("Fin del archivo de video alcanzado.")
63         break
64
65     # Convierte el fotograma a escala de grises
66     imagen_gris = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
67
68     # Busca los rostros de gatos

```

```

69     gatos = modeloGato.detectMultiScale(imagen_gris, scaleFactor=1.1, minNeighbors=6)
70
71     # Recorre el array 'gatos' y dibuja los rectángulos
72     for (x, y, ancho_rect, alto_rect) in gatos:
73         # Dibujamos en color rojo (0,0,255) con grosor 3
74         cv2.rectangle(img, (x, y), (x + ancho_rect, y + alto_rect), (0, 0, 255), 3)
75
76     # Guarda el fotograma modificado en el archivo de salida
77     grabador.write(img)
78
79     # Muestra el resultado en una ventana
80     cv2.imshow('Ejercicio 3 - Deteccion de Gatos', img)
81
82     # Condición de salida con la tecla ESC (27)
83     if cv2.waitKey(1) == 27:
84         print("Proceso interrumpido manualmente.")
85         break
86
87     # Libera recursos
88     camara.release()
89     grabador.release()
90     cv2.destroyAllWindows()
91     print(f"Video exportado correctamente como: '{video_salida}'\n")

```

Listing 41: Código completo del ejercicio3

5.8. Fichero *ejercicio4_img.py*

```

1  import cv2
2  import sys
3  import os
4
5  # Validación del número de argumentos en la línea de comandos
6  if len(sys.argv) < 2:
7      print("\n[ERROR] Uso correcto: python ejercicio4_img.py <nombre_imagen.jpg>\n")
8      sys.exit(1)
9
10 # Captura del parámetro del archivo de imagen
11 nombre_imagen = sys.argv[1]
12
13 # Carga el modelo clasificador de Haar
14 modeloCara = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
15 modeloCuerpo = cv2.CascadeClassifier('haarcascade_fullbody.xml')
16
17 # Verificación de que los modelos se han cargado correctamente
18 if modeloCara.empty():
19     print("Error: No se pudo cargar el clasificador de cara")
20     exit()
21 if modeloCuerpo.empty():
22     print("Error: No se pudo cargar el clasificador de cuerpo")
23     exit()
24
25 # Cargamos la imagen
26 img = cv2.imread(nombre_imagen)
27
28 # Control de excepciones ante rutas inexistentes o corruptas
29 if img is None:
30     print(f"[ERROR] No se pudo cargar la imagen '{nombre_imagen}'")
31     sys.exit(1)
32
33 # Procesamiento en escala de grises
34 imagen = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
35
36 # Buscamos los rostros:

```

```

37 rostros = modeloCara.detectMultiScale(imagen, scaleFactor=1.5, minNeighbors=6)
38
39 # Buscar los cuerpos:
40 cuerpos = modeloCuerpo.detectMultiScale(imagen, scaleFactor=1.3, minNeighbors=3)
41
42 # Dibujar rectángulos
43 for (x,y,ancho, alto) in rostros:
44     cv2.rectangle(img, (x,y), (x+ancho, y+alto), (0,0,255) , 3)
45 for (x,y,ancho, alto) in cuerpos:
46     cv2.rectangle(img, (x,y), (x+ancho, y+alto), (255,0,0) , 3)
47
48 # Guardar la imagen en disco
49 # Separamos el nombre del archivo de su extensión
50 nombre_base, extension = os.path.splitext(nombre_imagen)
51 # Creamos el nuevo nombre para el archivo de salida
52 nombre_salida = f"resultado_{nombre_base}{extension}"
53
54 # Guardamos la imagen modificada en la carpeta de trabajo
55 cv2.imwrite(nombre_salida, img)
56 print(f"Imagen resultante guardada con éxito como: {nombre_salida}")
57
58 #Abrimos una ventana con el resultado:
59 cv2.imshow(f'Output - {nombre_imagen}', img)
60 print("\nMostrando resultado del ejercicio 4. Pulsa cualquier tecla para salir.\n")
61
62 cv2.waitKey(0)
63 cv2.destroyAllWindows()

```

Listing 42: Código completo del ejercicio4 con imágenes

5.9. Fichero *ejercicio4_vid.py*

```

1 import cv2
2 import sys
3 import os
4
5 # Validación del número de argumentos en la línea de comandos
6 if len(sys.argv) < 2:
7     print("\n[ERROR] Uso correcto: python ejercicio4_vid.py <nombre_video.mp4>\n")
8     sys.exit(1)
9
10 # Captura del parámetro del archivo de imagen
11 nombre_video = sys.argv[1]
12
13 # Carga el modelo classificador de Haar
14 modeloCara = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
15 modeloCuerpo = cv2.CascadeClassifier('haarcascade_fullbody.xml')
16
17 # Verificación de que los modelos se han cargado correctamente
18 if modeloCara.empty():
19     print("Error: No se pudo cargar el clasificador de cara")
20     exit()
21 if modeloCuerpo.empty():
22     print("Error: No se pudo cargar el clasificador de cuerpo")
23     exit()
24
25 # Inicializa la captura del archivo de vídeo pasado por parámetro
26 camara = cv2.VideoCapture(nombre_video)
27
28 # Valida que el archivo realmente exista y OpenCV pueda decodificarlo
29 if not camara.isOpened():
30     print(f"\n[ERROR] No se pudo abrir el archivo de vídeo: '{nombre_video}'\n")
31     sys.exit(1)
32

```



```

33
34 # Extrae las propiedades nativas del vídeo original para que el de salida sea idé
    ntico
35 fps_leido = int(camara.get(cv2.CAP_PROP_FPS))      # Cuadros por segundo
36 ancho = int(camara.get(cv2.CAP_PROP_FRAME_WIDTH))  # Resolución: Ancho en píxeles
37 alto = int(camara.get(cv2.CAP_PROP_FRAME_HEIGHT))  # Resolución: Alto en píxeles
38
39 # Control de seguridad: Si los FPS no se leen bien, asignamos 30 por defecto para
    evitar el fallo
40 fps = int(fps_leido) if fps_leido > 0 else 30
41
42 # Guardar video de salida
43 # Obtenemos la ruta absoluta de la carpeta donde se está ejecutando el script
44 directorio_actual = os.path.dirname(os.path.abspath(__file__))
45
46 # Extraemos solo el nombre del archivo de vídeo (eliminando rutas previas si las hay)
47 nombre_archivo_puro = os.path.basename(nombre_video)
48 nombre_base, _ = os.path.splitext(nombre_archivo_puro)
49
50 # Combinamos todo en una ruta absoluta limpia para Windows
51 video_salida = os.path.join(directorio_actual, f"resultado_{nombre_base}.avi") #
    Guarda en formato AVI para máxima compatibilidad con códecs en Windows
52
53 # Definimos el códec de compresión de vídeo.
54 fourcc = cv2.VideoWriter_fourcc(*'XVID')
55
56 # Instanciamos el objeto grabador con la ruta, el códec, los FPS y el tamaño de los
    fotogramas
57 grabador = cv2.VideoWriter(video_salida, fourcc, fps, (ancho, alto), isColor=True)
58
59 # Validación de que el grabador se haya inicializado correctamente en el sistema
    operativo
60 if not grabador.isOpened():
61     print("[ERROR] No se pudo inicializar el grabador de vídeo. Revisa los permisos
        de la carpeta.")
62     sys.exit(1)
63
64 print(f"\n Procesando vídeo: '{nombre_video}' ({ancho}x{alto} a {fps} FPS)")
65 print(f"El resultado con las detecciones se guardará en: '{video_salida}'")
66 print("Presiona la tecla 'ESC' en la ventana gráfica para interrumpir el proceso.\n")
67
68 #Procesamiento frame a frame del vídeo
69 while camara.isOpened():
70     hay_frame, img = camara.read()
71
72     if not hay_frame:
73         print("Fin del archivo de vídeo alcanzado.")
74         break
75
76     # Convertimos el frame a escala de grises para el algoritmo de Viola-Jones
77     gris = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
78
79     # Buscar los rostros:
80     rostros = modeloCara.detectMultiScale(gris, scaleFactor=1.5, minNeighbors=6)
81
82     # Buscar los cuerpos:
83     cuerpos = modeloCuerpo.detectMultiScale(gris, scaleFactor=1.5, minNeighbors=3)
84
85     # Dibujar rectángulos
86     for (x,y,ancho, alto) in rostros:
87         cv2.rectangle(img, (x,y), (x+ancho, y+alto), (0,0,255) , 3) # Rojo
88     for (x,y,ancho, alto) in cuerpos:
89         cv2.rectangle(img, (x,y), (x+ancho, y+alto), (255,0,0) , 3) # Azul

```

```

90
91     print("Fin del procesamiento de vídeo. Guardando resultado final...")
92     # Guardar el video en disco
93     # Separamos el nombre del archivo de su extensión
94     nombre_base, extension = os.path.splitext(nombre_video)
95     # Creamos el nuevo nombre para el archivo de salida
96     nombre_salida = f"resultado_{nombre_base}{extension}"
97
98     # Guardamos el video modificado en la carpeta de trabajo
99     grabador.write(img)
100    print(f"Video resultante guardado con éxito como: {nombre_salida}")
101
102    #Abrimos una ventana con el resultado:
103    cv2.imshow(f'Ejercicio 4 - {nombre_video}', img)
104
105    # Escuchamos los eventos del teclado durante 1 milisegundo
106    key = cv2.waitKey(1)
107    # 27 es el código ASCII para la tecla 'Esc' (Escape)
108    if key == 27:
109        print("Proceso interrumpido manualmente por el usuario.")
110        break
111
112    # Liberar los descriptores para que Windows cierre correctamente los archivos de
113    # video
114    camara.release()
115    grabador.release()
116    cv2.destroyAllWindows()
117    print(f"Archivo de vídeo exportado correctamente en la raíz del proyecto.\n")

```

Listing 43: Código completo del ejercicio4 con video